

Desenvolvimento para Dispositivos Móveis

Material Design, Tabs, Bottom
Navigation, Drawer Navigation

Universidade Federal do Rio Grande do Norte

Material Design

- Introduzido pelo Google em 2014.
- É um sistema de design abrangente que unifica o visual, a experiência do usuário (UX), e os princípios de design de interface (UI).
- Baseado em princípios como profundidade, movimento e clareza.

Material Design

- Objetivos do Material Design
 - Criar uma linguagem visual consistente.
 - Proporcionar uma experiência unificada em diferentes dispositivos e plataformas.
 - Facilitar o desenvolvimento de interfaces bonitas e funcionais.

Princípios Fundamentais do Material Design

- **Metáfora do Material**
 - Elementos visuais como “materiais” que podem se mover, crescer, e mudar.
 - Usa sombras, luzes e perspectivas para criar uma sensação de profundidade.
- **Movimento Significativo**
 - Animações para guiar o usuário.
 - Movimentos suaves e intuitivos que reforçam a continuidade das ações.
- **Design Adaptável**
 - Interfaces responsivas que se adaptam a diferentes tamanhos de tela e dispositivos.
 - Uso de layouts dinâmicos e escaláveis.

Histórico

- Material Design 1 (2014)
 - Lançamento inicial com foco em design responsivo e animações.
- Material Design 2 (2018)
 - Introdução de temas dinâmicos, cantos arredondados e sombras mais sutis.
 - Maior ênfase em tipografia e espaçamento.
- Material Design 3 (2021)
 - Revolução com ênfase na personalização e acessibilidade.
 - Introdução de cores dinâmicas e novos componentes.

Aplicando o Material Design

- Especificações do Material Design podem ser encontradas no site <https://m3.material.io/>
- Em Flutter uma flag temporária que pode ser usada para desativar os recursos do Material 3.
 - Essa flag é verdadeira por padrão. Se definida como falsa, os componentes continuarão a usar as cores, tipografia e outros recursos do Material 2.
 - A longo prazo, essa flag será descontinuada e, eventualmente, apenas o Material 3 será suportado. É recomendado que as aplicações migrem para o Material 3 assim que for possível.

Aplicando o Material Design

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Flutter Demo',  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),  
        useMaterial3: true,  
      ),  
      home: const MyHomePage(title: 'Flutter Demo Home Page'),  
    );  
  }  
}
```

Aplicando o Material Design

- <https://docs.flutter.dev/ui/widgets/material>
- Prepare o tema da aplicação utilizando o Theme builder disponível na Web
- <https://material-foundation.github.io/material-theme-builder/>
- Baixe o tema criado e adicione ao projeto

Aplicando o Material Design

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Flutter Demo',  
      theme: const ThemeData(TextTheme()).light(),  
      darkTheme: const ThemeData(TextTheme()).dark(),  
      highContrastDarkTheme:  
        const ThemeData(TextTheme()).darkHighContrast(),  
      highContrastTheme: const ThemeData(TextTheme()).lightHighContrast(),  
      home: const MyHomePage(title: 'Flutter Demo Home Page'),  
    );  
  }  
}
```

Aplicando o Material Design

```
appBar: AppBar(  
  backgroundColor: Theme.of(context).colorScheme.inversePrimary,  
  title: Text(widget.title),  
)  
  
floatingActionButton: FloatingActionButton(  
  backgroundColor: Theme.of(context).colorScheme.tertiary,  
  onPressed: _incrementCounter,  
  tooltip: 'Increment',  
  child: Icon(Icons.add, color: Theme.of(context).colorScheme.onTertiary),  
)
```

Aplicando o Material Design

```
Text(  
  '$_counter',  
  style: Theme.of(context).textTheme.headlineMedium,  
)
```

Menus e Navegação

- O uso de Menus para organizar a navegação em Apps é uma atividade comum.
- Existem diversos tipos de menus. Muitos desses menus estão relacionados com interfaces antigas, por exemplo, o uso de abas.
- Tipos de menu
 - Abas
 - Navigation Drawer
 - Bottom Navigation

Abas

- **DefaultTabController** gerencia o estado e o comportamento das abas. Seu uso de dá envolvendo uma estrutura que inclui uma **TabBar** e uma **TabBarView**, fornecendo a eles um **TabController** compartilhado.
- **TabBar** exibe as abas interativas.
- **Tab** define cada aba individual.
- **TabBarView** exibe o conteúdo correspondente a cada aba selecionada.

Exemplo

```
return DefaultTabController(  
  length: 3,  
  child: Scaffold(  
    appBar: AppBar(  
      backgroundColor: Theme.of(context).colorScheme.primary,  
      title: Text("Aplicativo de Abas",  
        style: TextStyle(color: Theme.of(context).colorScheme.onPrimary),  
      ),  
    bottom: TabBar(  
      labelColor: Theme.of(context).colorScheme.onPrimary,  
      tabs: [  
        Tab(icon: Icon(Icons.ice_skating,  
          color: Theme.of(context).colorScheme.onPrimary),  
          text: "Tab 1" ),  
        Tab(text: "Tab 2"),  
        Tab(text: "Tab 3"),  
      ],),),  
    body: TabBarView(  
      children: [  
        Center(child: Text("Tab 1")),  
        Text("Tab 2"),  
        Text("Tab 3"),  
      ],),),),);
```



Abas

- Um `TabController` no Flutter é uma classe que gerencia o estado e o comportamento das abas (`Tab`) em uma interface de usuário que utiliza `TabBar` e `TabBarView`.
- Ele coordena a seleção de abas e sincroniza a mudança de abas entre a `TabBar` (onde as abas são exibidas) e a `TabBarView` (onde o conteúdo das abas é mostrado).

Abas

- O TabController mantém o estado da aba atualmente selecionada e notifica a interface de usuário quando há uma mudança de aba.
- O TabController garante que, quando uma aba é selecionada na TabBar, o conteúdo correspondente na TabBarView seja exibido.
- O TabController também gerencia a animação entre as abas, garantindo uma transição suave quando o usuário alterna entre diferentes abas.

Abas

- O `TabController` deve ser usado quando for necessário um controle detalhado sobre o comportamento das abas, como programaticamente selecionar uma aba ou ouvir mudanças de abas.

Abas

- Propriedades importantes

- `index`: Indica o índice da aba atualmente selecionada.
- `length`: Define o número total de abas que o controlador gerencia.
- `animateTo(int index)`: Anima a mudança de aba para o índice especificado.
- `previousIndex`: Armazena o índice da aba que estava selecionada anteriormente.

Exemplo

```

home: Scaffold(
  appBar: AppBar(
    bottom: TabBar(
      controller: _tabController,
      tabs: const [
        Tab(icon: Icon(Icons.directions_car)),
        Tab(icon: Icon(Icons.directions_transit)),
        Tab(icon: Icon(Icons.directions_bike)),
      ],
    ),
    title: const Text('Tabs Demo'),
  ),
  body: TabBarView(
    controller: _tabController,
    children: [
      const Icon(Icons.directions_car),
      Center(
        child: Column(
          children: [
            const Icon(Icons.directions_transit),
            TextButton(
              onPressed: () => _tabController.animateTo(0),
              child: const Text("Go to Aba 1"))
          ],
        ),
      ),
      const Icon(Icons.directions_bike),
    ],
  ),
),

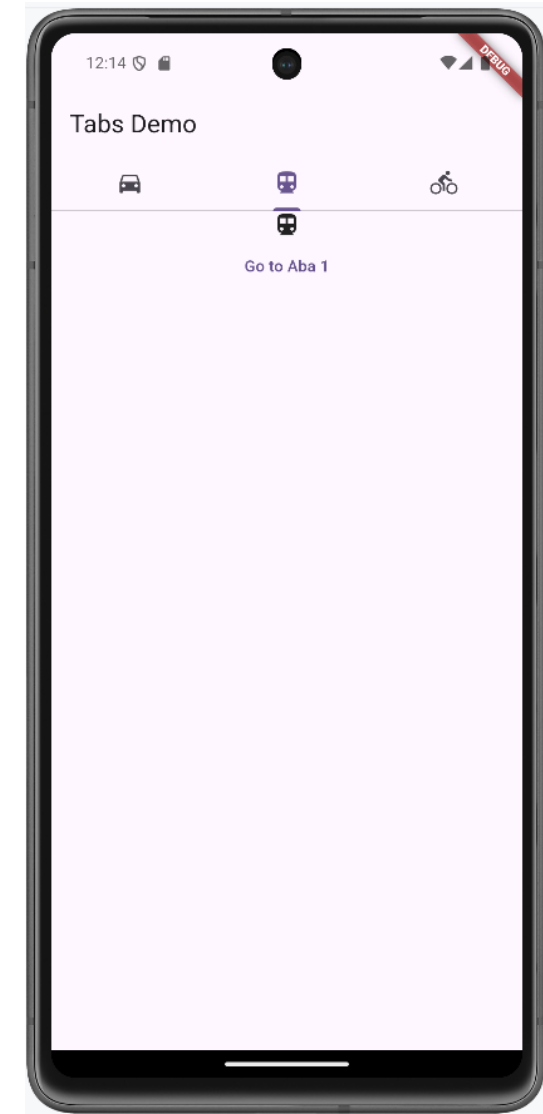
```

```

late TabController _tabController;

@override
void initState() {
  _tabController = TabController(
    length: 3, vsync: this);
  super.initState();
}

```



NavigationBar

- As widgets **NavigationBar** e **NavigationDestination** são usadas em conjunto para criar uma barra de **navegação na parte inferior da tela**, que permite ao usuário alternar entre diferentes seções de um aplicativo.
- Esse padrão de navegação é **comum em aplicativos móveis**, proporcionando uma maneira fácil e acessível de navegar entre diferentes telas.
- **ATENÇÃO: BottomNavigationBar e BottomNavigationBarItem estão em desuso e não devem ser usadas.**

NavigationBar

- NavigationBar é a widget que exibe a barra de navegação na parte inferior da tela. Ela pode conter vários itens (NavigationDestination), cada um representando uma aba ou seção diferente do aplicativo.
- É usada na definição do widget Scaffold para fornecer navegação entre diferentes seções de um aplicativo. Pode ser personalizada em termos de estilo e comportamento.

NavigationBar

- Propriedades importantes:
 - **destinations:** Uma lista de **NavigationDestination** que define os itens na barra de navegação.
 - **selectedIndex:** O índice do item atualmente selecionado.
 - **onDestinationSelect:** Um callback que é chamado quando um item é tocado, geralmente usado para mudar o estado e atualizar a interface de usuário.

NavigationDestination

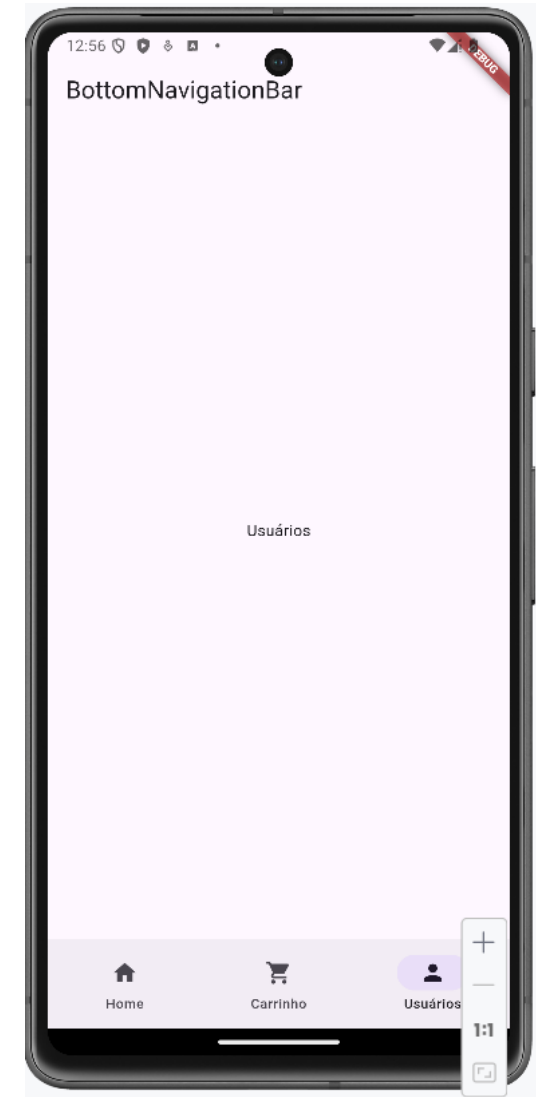
- **NavigationDestination** é a widget que define cada item individual dentro de uma `NavigationBar`. Cada item geralmente inclui um ícone e um rótulo (texto) que descrevem a função ou seção representada.
- Propriedades importantes:
 - **icon**: O ícone exibido para o item.
 - **label**: O texto associado ao item, que pode ser exibido abaixo do ícone.

Exemplo

```
class _HomeBottonNavState extends State<HomeBottonNav> {

  int _selectedIndex = 0;
  static const List<Widget> _widgetOptions = <Widget>[Text('Home'),Text('Carrinho'),Text('Usuários') ];
  void _onItemSelected(int index) {
    setState(() {
      _selectedIndex = index;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('BottomNavigationBar'),),
      body: Center(child: _widgetOptions.elementAt(_selectedIndex),),
      bottomNavigationBar: NavigationBar(
        destinations: const [
          NavigationDestination(icon: Icon(Icons.home), label: 'Home'),
          NavigationDestination(icon: Icon(Icons.shopping_cart),label: 'Carrinho'),
          NavigationDestination(icon: Icon(Icons.person),label: 'Usuários'),
        ],
        onDestinationSelected: _onItemSelected,
        selectedIndex: _selectedIndex,
      ),
    );
  }
}
```



NavigationDrawer

- As widgets `NavigationDrawer` e `DrawerHeader` no Flutter são usadas para criar uma barra lateral deslizante (drawer), que é comumente utilizada para navegação em aplicativos.
- Essa barra lateral pode conter itens de menu, informações do usuário, e outras opções que ajudam na navegação dentro do aplicativo.

NavigationDrawer

- Widget que representa uma barra lateral deslizante, geralmente usada para navegação. Ela pode ser deslizada a partir da borda esquerda (ou direita, dependendo da direção do texto) da tela.
- Normalmente usada com o widget **Scaffold** para fornecer uma navegação alternativa dentro de um aplicativo, permitindo ao **usuário acessar diferentes partes do aplicativo a partir de uma lista de opções**.

NavigationDrawer

- **Propriedades Importantes**
 - **children**: Define o conteúdo do drawer, que geralmente inclui array widgets representando os itens de navegação.
 - **backgroundColor**: Define a cor de fundo da drawer.

DrawerHeader

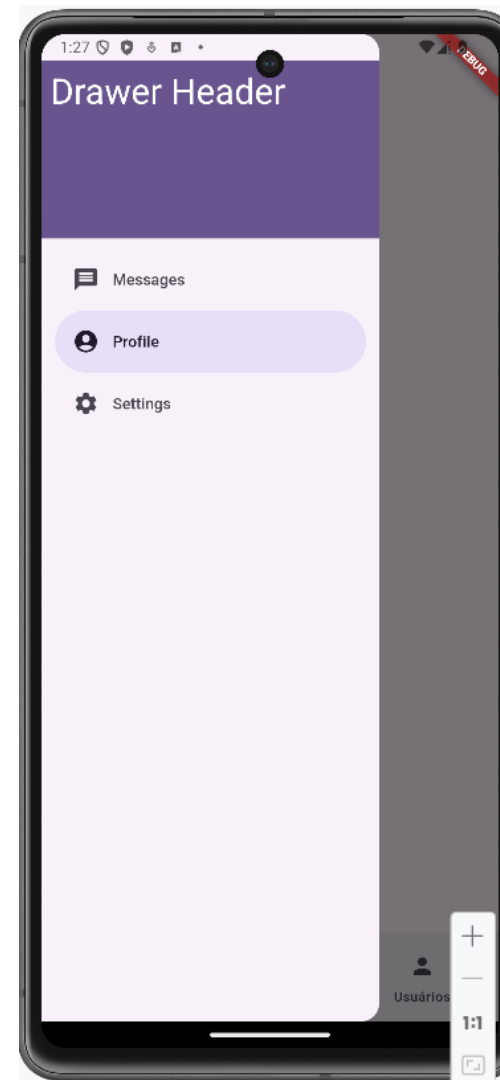
- DrawerHeader é uma widget que normalmente aparece na parte superior do Drawer e é usada para mostrar informações contextuais, como o nome do usuário, uma foto de perfil, ou o título do menu de navegação.
- É comumente usada para fornecer um cabeçalho estilizado no NavigationDrawer, que pode conter uma combinação de textos, imagens, ícones e outros widgets para criar uma seção de introdução ou identidade.

DrawerHeader

- **Propriedades Importantes:**
 - **decoration:** Permite definir uma decoração personalizada, como uma cor de fundo, uma imagem ou um gradiente.
 - **child:** Define o conteúdo dentro do DrawerHeader, que pode ser um texto, uma imagem, ou qualquer outra widget.
 - **padding:** Define o espaçamento interno dentro do DrawerHeader.

Exemplo

```
class _HomeBottonNavState extends State<HomeBottonNav> {  
  int _selectedIndex = 0;  
  static const List<Widget> _widgetOptions = <Widget>[ Text('Home'), Text('Carrinho'), Text('Usuários') ];  
  void handleScreenChanged(int selectedScreen) {  
    setState(() {  
      _selectedIndex = selectedScreen;  
    });  
  }  
  Widget build(BuildContext context) {  
    return Scaffold(  
      body: Center(child: _widgetOptions.elementAt(_selectedIndex)),  
      drawer: NavigationDrawer(  
        onDestinationSelected: handleScreenChanged,  
        selectedIndex: _selectedIndex,  
        children: [  
          DrawerHeader(  
            decoration: BoxDecoration(color: Theme.of(context).colorScheme.primary),  
            child: Text(  
              'Drawer Header',  
              style: Theme.of(context).textTheme.headlineLarge!.copyWith(  
                color: Theme.of(context).colorScheme.onPrimary,  
              )),),  
          const NavigationDrawerDestination(  
            icon: Icon(Icons.message),  
            label: Text('Messages'),  
          ),  
          const NavigationDrawerDestination(  
            icon: Icon(Icons.account_circle),  
            label: Text('Profile'),  
          ),  
          const NavigationDrawerDestination(  
            icon: Icon(Icons.settings),  
            label: Text('Settings'),  
          ),  
        ],),),);  
  }
```



Referências

- <https://docs.flutter.dev/>